

University of Sunderland

CET341 – Assignment 2 Report

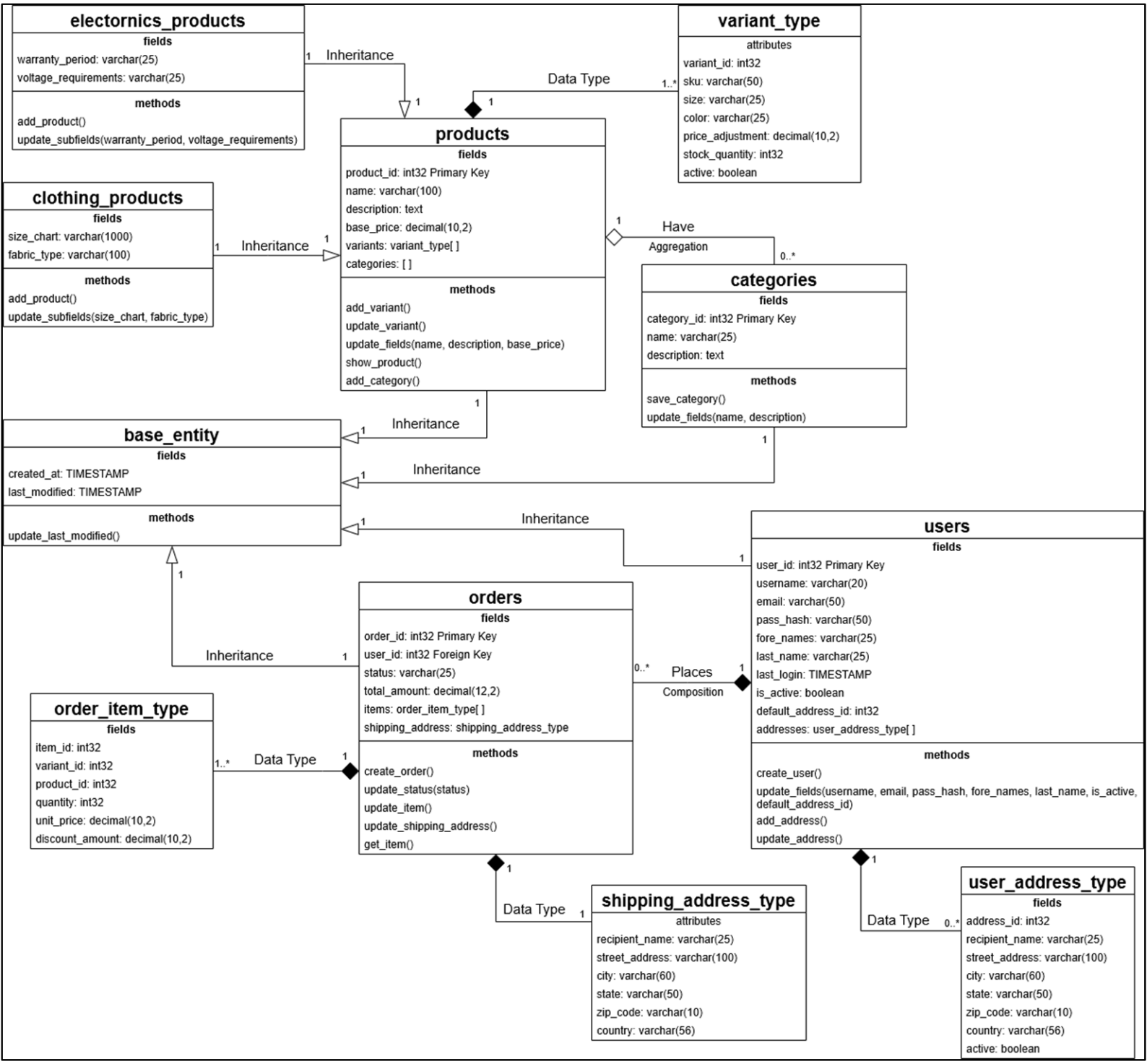
A brief comparison of SQL and NoSQL databases using PostgreSQL and MongoDB with an e-commerce database

Name: Syed Hani Haider Naqvi

Student No: 240539894

University of Sunderland

Task One:



The above UML diagram represents a part of an e-commerce platform. I worked on a project during my HND to build an e-commerce store app. This database is inspired by that project, but I've decided to build a skeleton model with only the core entities needed to complete the tasks of this project, as a full database for a project like Shopify would be very huge and too complicated. There are mainly 4 entities in the PostgreSQL and MongoDB databases. The Products, categories, orders, and users. All of them inherit temporal fields from the base entity. The base

entity in the original design was for creating event logs, but it only serves as a single place to inherit temporal data from in this design. There are 4 composite data types/nested documents in this design. A user_address for storing an array of addresses that the user saves, a shipping_address to ensure past orders data doesn't change in the records in case the user changes an address. There is an order_item type for storing order items with IDs that refer to products and variants, and other data related to the order item. There is a variant type for products to store an array of variants. Lastly, the products are divided into subtypes to ensure specific product-related fields are added only to the correct product, and there aren't too many null fields in the records. The methods listed in the entities would be built for production implementation, but only the ones needed for this demonstration are built in the submitted database.

Task Two:

The many-to-many relationship in PostgreSQL is done with a junction table as the other option would have been to add all the categories to the products and all the products to the categories table (Strausz, 2023). There is a direct 1-1 link between products and categories for the primary category of the product. All the SQL commands are in uppercase to allow better readability of the script.

Creating types:

The types are dropped (with all the tables and triggers by the CASCADE clause to prevent dependency errors) and created successfully.

Running the entire type block returns this:

```
NOTICE:  drop cascades to column item of table orders
NOTICE:  drop cascades to 4 other objects
NOTICE:  drop cascades to column shipping_address of table orders
NOTICE:  drop cascades to column addresses of table users
CREATE TYPE

Query returned successfully in 82 msec.
```

```
-- USER DEFINED TYPES

DROP TYPE IF EXISTS order_item_type CASCADE;
DROP TYPE IF EXISTS variant_type CASCADE;
DROP TYPE IF EXISTS shipping_address_type CASCADE;
DROP TYPE IF EXISTS address_type CASCADE;

-- Soft deletion is implemented with "active" fields

CREATE TYPE address_type AS (
    address_id INTEGER,
    recipient_name varchar(25),
    street VARCHAR(100),
    city VARCHAR(60),
    state VARCHAR(50),
    zip_code VARCHAR(10),
    country VARCHAR(56),
    active BOOLEAN);

CREATE TYPE shipping_address_type AS (
    recipient_name varchar(25),
    street VARCHAR(100),
    city VARCHAR(60),
    state VARCHAR(50),
    zip_code VARCHAR(10),
    country VARCHAR(56));

CREATE TYPE variant_type AS (
    variant_id INTEGER,
    sku VARCHAR(50),
    size VARCHAR(25),
    color VARCHAR(25),
    price_adjustment DECIMAL(10,2),
    stock_quantity INTEGER,
    active BOOLEAN);

-- The orders table has a slightly different address
CREATE TYPE order_item_type AS (
    item_id INTEGER,
    variant_id INTEGER,
    product_id INTEGER,
    quantity INTEGER,
    unit_price DECIMAL(10,2),
    discount_amount DECIMAL(10,2));
```

Creating Tables:

All the tables are dropped by the types as dependencies, but the drop statements are there, and the tables are created successfully. The pets_products table is there to showcase an unpublished product and has no stock or categories linked to it.

```
DROP TABLE IF EXISTS orders CASCADE;
DROP TABLE IF EXISTS products_categories_links CASCADE;
DROP TABLE IF EXISTS categories CASCADE;
DROP TABLE IF EXISTS pet_products CASCADE;
DROP TABLE IF EXISTS electronics_products CASCADE;
DROP TABLE IF EXISTS clothing_products CASCADE;
DROP TABLE IF EXISTS products CASCADE;
DROP TABLE IF EXISTS users CASCADE;
DROP TABLE IF EXISTS base_entity CASCADE;

-- Base entity table for inheriting temporal data to all
CREATE TABLE base_entity (
    entity_id SERIAL PRIMARY KEY,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    last_modified TIMESTAMP DEFAULT CURRENT_TIMESTAMP);

-- User table. The default_address_id is meant for fetchi
CREATE TABLE users (
    user_id INTEGER NOT NULL PRIMARY KEY,
    username VARCHAR(20) UNIQUE NOT NULL,
    email VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(50) NOT NULL,
    forenames VARCHAR(25),
    last_name VARCHAR(25) NOT NULL,
    last_login TIMESTAMP,
    is_active BOOLEAN DEFAULT TRUE,
    default_address_id INTEGER,
    addresses address_type[]) INHERITS (base_entity);
```

```
-- Products has three subtypes for this demonstration, only two are published and have
CREATE TABLE products (
    product_id INTEGER NOT NULL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    base_price DECIMAL(10,2) NOT NULL,
    variants variant_type[] NOT NULL) INHERITS (base_entity);

CREATE TABLE clothing_products (
    size_chart VARCHAR(1000) NOT NULL,
    fabric_type VARCHAR(100) NOT NULL) INHERITS (products);

CREATE TABLE electronics_products (
    warranty_period VARCHAR(25) NOT NULL,
    voltage_requirements VARCHAR(25) NOT NULL) INHERITS (products);

CREATE TABLE pet_products (
    pet_type VARCHAR(50) NOT NULL,
    material VARCHAR(100) NOT NULL) INHERITS (products);

-- There is only one categories table to keep things simple and within the given limits,
CREATE TABLE categories (
    category_id INTEGER NOT NULL PRIMARY KEY,
    name VARCHAR(25) NOT NULL,
    description TEXT) INHERITS (base_entity);

-- The following table holds relations between products and categories. This way multipl
CREATE TABLE products_categories_links (
    product_id INTEGER NOT NULL ON DELETE CASCADE,
    category_id INTEGER NOT NULL REFERENCES categories(category_id) ON DELETE CASCADE,
    is_primary BOOLEAN DEFAULT FALSE) INHERITS (base_entity);

-- This last order table is a bit basic but I did not want to complicate things more tha
CREATE TABLE orders (
    order_id INTEGER NOT NULL PRIMARY KEY,
    user_id INTEGER NOT NULL REFERENCES users(user_id),
    item order_item_type NOT NULL,
    status VARCHAR(25) NOT NULL,
    total_amount DECIMAL(12,2),
    shipping_address shipping_address_type NOT NULL) INHERITS (base_entity);
```

Running the entire table block returns this:

```
CREATE TABLE

Query returned successfully in 60 msec.
```

Creating Procedure:

There is one procedure used to update a field in an array for Query C. It is dropped, then created successfully. The procedure takes 3 arguments. The first one is the product_id to find the product, the second is the variant_id to find the variant, and the third is the new price that will be updated with the procedure. It finds the variant in a WITH clause and uses FOR UPDATE to prevent any other transactions from using it for the duration of this transaction. There are proper constraints in place to raise exceptions on null values and negative prices.

```
-- STORED PROCEDURES

DROP PROCEDURE IF EXISTS update_variant_price;

-- Prodedure for updating additional price of a variant
CREATE OR REPLACE PROCEDURE update_variant_price
(
  pid INTEGER,
  vid INTEGER,
  price DECIMAL(10,2)
)
AS
$$
BEGIN

  IF pid IS NULL THEN
    RAISE EXCEPTION 'product_id must have a value';
  END IF;
  IF vid IS NULL THEN
    RAISE EXCEPTION 'vairant_id must have a value';
  END IF;
  IF price IS NULL THEN
    RAISE EXCEPTION 'price can not be null';
  END IF;
    IF price < 0 THEN
        RAISE EXCEPTION 'price can not be negative';
    END IF;

  WITH variant_for_update AS (
  SELECT
  p.product_id AS p_id,
  p.variants[vid].variant_id AS v_id
  FROM products p
  WHERE p.product_id = pid
  FOR UPDATE)

  UPDATE products
  SET variants[variant_for_update.v_id].price_adjustment = price
  FROM variant_for_update
  WHERE products.product_id = variant_for_update.p_id;

END;
$$
LANGUAGE plpgsql;
```

Running the stored procedure block returns this:

```
CREATE PROCEDURE

Query returned successfully in 92 msec.
```

Creating Functions:

There are two functions, one used in Query A and one for the triggers. They are dropped and created successfully. The first function is to fetch the user who has ordered a variant of a product the most frequently. It takes the product_id and the variant_id as arguments. The second one is to update the last_modified timestamp on the products table, and the update is triggered.

```
-- STORED FUNCTIONS

DROP FUNCTION IF EXISTS update_last_modified CASCADE;
DROP FUNCTION IF EXISTS most_frequent_buyer;

-- Function to get the most frequent buyer of a single product variant
CREATE OR REPLACE FUNCTION most_frequent_buyer(pid INTEGER, vid INTEGER)
RETURNS INTEGER
AS
$$
DECLARE uid INTEGER;
BEGIN
  SELECT o.user_id INTO uid
  FROM orders o
  WHERE (o.item).product_id = pid AND (o.item).variant_id = vid
  GROUP BY o.user_id
  ORDER BY count(o.user_id) DESC, o.user_id
  LIMIT 1;
  RETURN uid;
END;
$$
LANGUAGE plpgsql;

-- Trigger Function
CREATE OR REPLACE FUNCTION update_last_modified()
RETURNS TRIGGER
AS
$$
BEGIN
    NEW.last_modified := CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$
LANGUAGE plpgsql;
```

Running the functions block of the script returns this:

```
CREATE FUNCTION

Query returned successfully in 56 msec.
```

Creating Triggers:

There are three update triggers for the update procedure. They are dropped when the types are dropped, as they have dependencies, but they have drop statements as well and are created successfully every time.

```
-- TRIGGERS

DROP TRIGGER IF EXISTS clothing_prod_update ON clothing_products;
DROP TRIGGER IF EXISTS electronics_prod_update ON electronics_products;
DROP TRIGGER IF EXISTS pet_prod_update ON pet_products;

CREATE OR REPLACE TRIGGER pet_prod_update
BEFORE UPDATE ON pet_products
FOR EACH ROW
EXECUTE PROCEDURE update_last_modified();

CREATE OR REPLACE TRIGGER electronics_prod_update
BEFORE UPDATE ON electronics_products
FOR EACH ROW
EXECUTE PROCEDURE update_last_modified();

CREATE OR REPLACE TRIGGER clothing_prod_update
BEFORE UPDATE ON clothing_products
FOR EACH ROW
EXECUTE PROCEDURE update_last_modified();
```

Running the procedure block returns this:

```
NOTICE:  trigger "clothing_prod_update" for relation "clothing_products" does not exist, skipping
NOTICE:  trigger "electronics_prod_update" for relation "electronics_products" does not exist, skipping
NOTICE:  trigger "pet_prod_update" for relation "pet_products" does not exist, skipping
CREATE TRIGGER

Query returned successfully in 56 msec.
```

Inserting sample data:

There are 22 users, 15 products (5 clothing, 5 electronics, 5 pets), 15 categories, and 200 orders in the sample data set. All are inserted successfully. There is a single insert command for each table instead of an insert command for each individual value.

```
INSERT 0 200

Query returned successfully in 99 msec.
```

The entire script to create all the objects and insert the sample data takes about 100-110 milliseconds on average.

```
NOTICE:  drop cascades to column item of table orders
NOTICE:  drop cascades to 4 other objects
NOTICE:  drop cascades to column shipping_address of table orders
NOTICE:  drop cascades to column addresses of table users
NOTICE:  trigger "clothing_prod_update" for relation "products" does not exist, skipping
NOTICE:  trigger "electronics_prod_update" for relation "products" does not exist, skipping
NOTICE:  trigger "pet_prod_update" for relation "products" does not exist, skipping
INSERT 0 200

Query returned successfully in 103 msec.
```

Task Three:

The MongoDB script starts with a db = connect() to create a database and then it mainly has db.collection.insertMany commands and a createIndex command below each collection. Indexes are created on user_id, product_id, category_id, and order_id fields in the users, products, categories, and orders collections, respectively. This is to emulate how indexes would be created when switching from a relational database to MongoDB with old data. Hypothetically, after deployment, all the documents would get the new auto-generated _id in the referenced id fields, but as I used the same data, I kept the IDs from the SQL database for simplicity. Of course, the collections are modified to reduce the number of collections (e.g. the junction table for products x categories is gone) and denormalise data where it makes sense. As the documents in JSON format take up thousands of lines, I will just attach unique examples from each document.

Creating/connecting DB:

The script first attempts to connect to the DB or creates it if it doesn't exist. Right after that, a for loop checks for the collections and drops them if they exist:

```
// change the name of the localhost if needed. This would create
db = connect ('mongodb://localhost/bi91zm-CET341-A2');


// Simple function to drop the collections if they exist
const collections= ["users","products","categories","orders"];
collections.forEach(collectionName => {
  if (db.getCollectionNames().includes(collectionName)) {
    db[collectionName].drop();
    print('Dropped existing collection:', collectionName)
  } else {
    print('Collection', collectionName, 'did not exist');
  }
});
```

```
PS D:\...\3. MongoDB script>mongosh --file '.\M
Collection users did not exist
Collection products did not exist
Collection categories did not exist
Collection orders did not exist
```

```
PS D:\...\3. MongoDB script>mongosh --file '.\M
Dropped existing collection: users
Dropped existing collection: products
Dropped existing collection: categories
Dropped existing collection: orders
```

Users:

The user collection has pretty much the same format as the PostgreSQL table, as changes were not needed here. The collection is created and validation rules are added first, then indexes, and then the data is inserted. I will only show one example of the user document to showcase all the fields as the only difference is the number of addresses between users and fields are the same across all documents:

```
db.createCollection("users", {
  validator: {
    $jsonSchema: {
    bsonType: "object",
    required: ["username","email","password_hash","last_name","is_active",],
    properties: {
      username: {
        bsonType: "string",
        maxLength: 20,
        description: "Username is required and must be a unique string"
      },
      email: {
        bsonType: "string",
        maxLength: 50,
        description: "Email is required and must be a unique string"
      },
      password_hash: {
        bsonType: "string",
        maxLength: 50,
        description: "Password is required and must be a hashed string"
      },
      forenames: {
        bsonType: "string",
        maxLength: 25,
        description: "Forenames should be string value"
      },
      last_name: {
        bsonType: "string",
        maxLength: 25,
        description: "Last name is required and must be a string"
      },
      is_active: {
        bsonType: "bool",
        description: "Account status must be a boolean"
      }
    }
  }
  }
});

db.users.createIndex( { user_id: 1, username: 1, email: 1 }, { unique: true });
```

```
db.users.insertMany([
  {
    "user_id": 1,
    "username": "john_smith",
    "email": "smith.j@domain.com",
    "password_hash": "a1b2c3d4e5f6g7h8i9j0",
    "forenames": "John",
    "last_name": "Smith",
    "last_login": ISODate("2025-02-15T10:30:00Z"),
    "is_active": true,
    "default_address_id": 2,
    "addresses": [
      {
        "address_id": 1,
        "recipient_name": "John Smith",
        "street": "10 Downing Street",
        "city": "London",
        "state": "England",
        "zip_code": "SW1A 2AA",
        "country": "UK",
        "active": false
      },
      {
        "address_id": 2,
        "recipient_name": "John Smith",
        "street": "22 Park Lane",
        "city": "London",
        "state": "England",
        "zip_code": "SW1A 2AA",
        "country": "UK",
        "active": true
      }
    ],
    "created_at": ISODate("2024-06-06T09:15:22Z"),
    "last_modified": ISODate("2024-06-09T17:13:55Z")
  },
```

Products:

There is only one collection for products, unlike the three tables for each type. The product collection has 2 more attributes than the SQL tables did, and those are product_type (for specifying which type of product it is) and the primary category's name. The specific product type/subtable-based fields are added to their respective document, so there are 3 different kinds of documents in this collection. I have attached a screenshot of the validations and index, and one document of each type. Each product has 6 variations. I have only kept 2, expanded to reduce the size, the fields are standard in all variations.

```
db.createCollection("products", {
  validator: {
    $jsonSchema: {
    bsonType: "object",
    required: ["product_type","name","base_price","variants"],
    properties: {
      product_type: {
        bsonType: "string",
        maxLength: 25,
        description: "The product type is required"
      },
      name: {
        bsonType: "string",
        maxLength: 100,
        description: "Product name is required and must be a uniqu
      },
      description: {
        bsonType: "string",
        description: "Description should be a string"
      },
      base_price: {
        bsonType: "double",
        description: "base_price is required and should be a decim
      },
      variants: {
        bsonType: "array",
        description: "Atleast one variant must be added to the prc
      }
    }
  }
  }
});

db.products.createIndex( { product_id: 1 }, { unique: true });
```

```
db.products.insertMany([
// ================== CLOTHING PRODUCTS ==================
  {
    "product_id": 101,
    "product_type": "clothing",
    "name": "Mens Jeans",
    "description": "This is a jeans for men",
    "base_price": 14.99,
    "variants": [
      {
        "variant_id": 1,
        "sku": "INV1001",
        "size": "Small",
        "color": "White",
        "price_adjustment": 0.00,
        "stock_quantity": 47,
        "active": true
      },
      {…
      },
      {
        "variant_id": 3,
        "sku": "INV1003",
        "size": "Medium",
        "color": "White",
        "price_adjustment": 1.00,
        "stock_quantity": 65,
        "active": true
      },
      {…
      },
      {…
      },
      {…
      }
    ],
    "size_chart": "Small: 30 Waist: 80cm Seat: 100cm Thigh: 56cm\nMedium: 35 Wais
    "fabric_type": "Jeans",
    "created_at": ISODate("2024-05-25T09:15:00Z"),
    "last_modified": ISODate("2025-03-10T14:30:00Z"),
    "primary_category_id": 2,
    "primary_category": "Mens Clothing"
  },
```

```
// ================== ELECTRONICS PRODUCTS ==================
  {
    "product_id": 201,
    "product_type": "electronics",
    "name": "iPhone Charger",
    "description": "This is an iPhone charger",
    "base_price": 24.99,
    "variants": [
      {
        "variant_id": 1,
        "sku": "INV1031",
        "color": "Black",
        "price_adjustment": 0.00,
        "stock_quantity": 42,
        "active": true
      },
      {…
      },
      {…
      },
      {
        "variant_id": 4,
        "sku": "INV1034",
        "color": "Emerald Green",
        "price_adjustment": 0.00,
        "stock_quantity": 19,
        "active": true
      },
      {…
      },
      {…
      }
    ],
    "warranty_period": "6 months",
    "voltage_requirements": "110V - 240V 60W",
    "created_at": ISODate("2024-05-26T10:15:00Z"),
    "last_modified": ISODate("2024-11-12T14:30:00Z"),
    "primary_category_id": 7,
    "primary_category": "Electronic Accessories"
  },
```

```
// ================== PET PRODUCTS ==================
  {
    "product_id": 301,
    "product_type": "pets",
    "name": "Cat Food",
    "description": "This is a great choice of food for a healthy cat",
    "base_price": 4.99,
    "variants": [
      {
        "variant_id": 1,
        "sku": "INV1061",
        "size": "Kitten 0.5 kg",
        "price_adjustment": 0.00,
        "stock_quantity": 0,
        "active": false
      },
      {…
      },
      {…
      },
      {
        "variant_id": 4,
        "sku": "INV1064",
        "size": "Adult 1.0 kg",
        "price_adjustment": 3.00,
        "stock_quantity": 0,
        "active": false
      },
      {…
      },
      {…
      }
    ],
    "pet_type": "Cats",
    "material": "Fish, Chicken, Beef, Vegetables",
    "created_at": ISODate("2025-05-15T08:00:00Z"),
    "last_modified": ISODate("2025-05-16T09:15:22Z"),
    "primary_category_id": null,
    "primary_category": null
  },
```

Categories:

The categories are essentially the same with the addition of one attribute, an array of the product IDs linked to the category has been passed as the last field in this collection. As the category documents are small, I was able to get two in one screenshot with the validations and the index as well.

```
db.createCollection("categories", {
  validator: {
    $jsonSchema: {
    bsonType: "object",
    required: ["name"],
    properties: {
      name: {
        bsonType: "string",
        maxLength: 25,
        description: "Category name is required and must be a unique string"
      }
    }
  }
  }
});

db.categories.createIndex( { category_id: 1 }, { unique: true });

db.categories.insertMany([
  {
    "category_id": 1,
    "name": "Clothing",
    "description": "These are all of the clothing items available on the store",
    "created_at": ISODate("2024-05-27T09:30:00Z"),
    "last_modified": ISODate("2024-05-27T09:30:00Z"),
    "products": [101, 102, 103, 104, 105]
  },
  {
    "category_id": 2,
    "name": "Mens Clothing",
    "description": "These clothing items are for men",
    "created_at": ISODate("2024-05-27T09:45:00Z"),
    "last_modified": ISODate("2024-05-27T09:45:00Z"),
    "products": [101, 103]
  },
```

Orders:

Order collection is essentially the same, with a key addition of the user's full name in the user. This addition is made to improve efficiency and avoid extra $lookup aggregations for a more optimised pipeline. I have included one document example, as all the documents follow the same pattern.

```
db.createCollection("orders", {
  validator: {
    $jsonSchema: {
    bsonType: "object",
    required: ["user","item","status","total_amount","shipping_address"],
    properties: {
      user: {
        bsonType: "string",
        maxLength: 50,
        description: "The name of the user is required"
      },
      item: {
        bsonType: "object",
        description: "An item must exist in an order"
      },
      status: {
        bsonType: "string",
        maxLength: 25,
        description: "Order must have a status e.g. Pending Payment, Processing, Shipped, Delivered, etc."
      },
      total_amount: {
        bsonType: "decimal",
        description: "There should be an order total e.g. 24.86"
      },
      shipping_address: {
        bsonType: "object",
        description: "An order must have a shipping address"
      },
    }
  }
  }
});

db.orders.createIndex( { order_id: 1 }, { unique: true });
```

```
db.orders.insertMany([
  {
    "order_id": 1,
    "user_id": 1,
    "user": "John Smith",
    "item": {
      "item_id": 1,
      "variant_id": 3,
      "product_id": 101,
      "quantity": 1,
      "unit_price": NumberDecimal("15.99"),
      "discount_amount": NumberDecimal("0.00")
    },
    "status": "delivered",
    "total_amount": NumberDecimal("15.99"),
    "shipping_address": {
      "recipient_name": "John Smith",
      "street": "10 Downing Street",
      "city": "London",
      "state": "England",
      "zip_code": "SW1A 2AA",
      "country": "UK"
    },
    "created_at": ISODate("2024-06-06T09:15:22Z"),
    "last_modified": ISODate("2024-06-13T09:15:22Z")
  },
```

Task Four:

There are quite a few key differences in my implementation of NoSQL and SQL databases. The main differences are the reduced entities in the NoSQL DB as inheritance is done through nested documents, and the data is denormalised. The main reason for denormalising the data (e.g. adding the user's name in orders, adding child-specific fields in just one product collection, etc) was to improve performance since it would reduce the number of $lookup stages in aggregation pipelines and joins in NoSQL can be very resource intensive (MongoDB Docs, no date). I also made sure all the fields used in $lookup were declared as indexes to improve the performance of the aggregation pipelines (Gupta, 2024).

I have removed the base entity as it was easier and faster to inherit it into other tables than adding it manually, and the base_entity table could be altered later to hold more data that could help generate system logs, however, having a separate collection for holding temporal data would have drastically affected the performance of MongoDB whenever a temporal restraint had to be used in an aggregation pipeline. I also removed the child entities of products, as MongoDB offers a flexible schema, and the child-specific field could just be added to the main products entity. To ensure that it was easier to know which child-specific fields were in which document, I enforced the product_type field through schema validators, and a new product_type can be sent from the backend to add another type of product instead of creating a new child table for it. Furthermore, I did not need to add a collection for the junction table, which holds many-many relationships in SQL, as I could just pass an array of products inside categories and the name of the primary category inside products. In SQL, foreign key fields can only hold one value, and foreign keys can not be inside arrays, but relations are not established through keys in NoSQL, so the product IDs could just be saved inside an array. Instead of having 9 collections for each table, I was able to get that down to 4 collections by denormalising some data and using NoSQL features.

I made as many validation rules and unique indexes as needed for this demonstration (it is not an exact schema match with the SQL DB), but of course, there would be a lot more rules built in the backend of the application as well in production. Building all the constraints and validation rules in the database can increase the database maintenance workload. Most database constraints are going to be in application logic anyway, but having duplicate validations for everything can increase the resources and cost of running things. It would be best to build constraints for the fields that must remain unique or used in relations or performing tasks where a certain value is expected in the database, and keep the

rest of the validation rules on the application side to bring down the cost of running things. Engineers should strive to build better logic rather than needing safeguards.

Tasks Five and Six:

Query a: A join of three or more tables – you should consider various types of join in this query (e.g. inner join, left/right/full outer joins, etc.) and the query must include a restriction on the rows selected

This query shows the total sales for each variant, their category, and the user who purchased it most frequently (with the help of the stored function in SQL). It is very helpful to see the best-performing variants in your store. The restrictions are applied on the joined entities to avoid getting unnecessary data (one example is the limit clause in the stored function to only get the most frequent user and not every user that has purchased an item)

In this query, the variants array is unnested and cross-joined with products to create a row for each variant.
Then orders have a left join to show a count for and the accumulated total sales for each variant, without removing the ones that do not have any sales.
After that, the primary categories for the products are fetched with an inner join to restrict products that have a category, as unpublished products are not assigned to a category (two joins as the relational data is stored in a junction table).
Lastly, the most frequent users are added with a left join to not exclude any items that have not been purchased.

In the aggregation pipeline, users are not joined as the user's full name is already in the orders collection, but the orders are looked up twice, once for the count and once for the user's full name.

SQL code

```
SELECT
p.product_id AS id,
p.name AS name,
CONCAT(v.size, ' | ', v.color) AS variant_details,
count(o.*) AS orders,
sum(o.total_amount) AS revenue,
c.name AS main_category,
CONCAT(u.forenames, ' ' ,u.last_name) AS most_purchased_by

FROM products p
```

```
CROSS JOIN unnest(p.variants) AS v

LEFT JOIN orders o ON v.variant_id = (o.item).variant_id AND p.product_id =
(o.item).product_id

JOIN products_categories_links pc ON p.product_id = pc.product_id AND
pc.is_primary = TRUE
JOIN categories c ON pc.category_id = c.category_ID

LEFT JOIN users u ON most_frequent_buyer(p.product_id, v.variant_id) =
u.user_id

GROUP BY p.product_id, v.variant_id, v.size, v.color, c.name, u.forenames,
u.last_name

ORDER BY count(o.*) DESC, p.product_id, v.variant_id;
```

MongoDB code

```
db.products.aggregate([{
   $match: { primary_category_id: { $exists: true, $ne: null }}
 },
 { $unwind: {
    path: "$variants",
    preserveNullAndEmptyArrays: true
  }},
 { $lookup: {
    from: "orders",
    localField: "product_id",
    foreignField: "item.product_id",
    let: {vid: "$variants.variant_id"},
    pipeline: [{
       $match: { $expr: { $eq: ["$item.variant_id", "$$vid"]}}},
     { $group: {
        _id: null,
        count: { $sum: 1 },
        revenue: { $sum: "$total_amount" }}}],
    as: "orders"
  }},
 { $unwind: {
    path: "$orders",
    preserveNullAndEmptyArrays: true }
```

```
},
{ $sort: {
   "orders.count": -1,
   product_id: 1,
   "variants.variant_id": 1 }
},
{ $lookup: {
   from: "orders",
   localField: "product_id",
   foreignField: "item.product_id",
   let: { vid: "$variants.variant_id" },
   pipeline: [{
      $match: { $expr: { $eq: ["$item.variant_id", "$$vid"]}}},
    { $group: {
       _id: {
         user_id: "$user_id",
         user: "$user"
       },
       count: { $sum: 1 }}
    },
    { $sort: {
       count: -1,
       "_id.user_id": 1 }
    },
    { $limit: 1
    },
    { $project: {
       _id: 0,
       name: "$_id.user"}
    }],
   as: "most_freq_user"
 }},
{ $unwind: {
   path: "$most_freq_user",
   preserveNullAndEmptyArrays: true }
},
{ $project:
   {
     _id: 0,
     product_id: "$product_id",
     name: "$name",
```

```
      variant_details: { $cond: { if: "$variants.size", then: { $cond: { if:
"$variants.color", then: { $concat: [ "$variants.size", " | ", "$variants.color" ]},
else: { $concat: [ "$variants.size", " | " ]}}}, else: { $concat: [" | ",
"$variants.color"]}}},
      orders: "$orders.count",
      revenue: "$orders.revenue",
      main_category: "$primary_category",
      most_purchased_by: "$most_freq_user.name"
    }
  }
]);
```

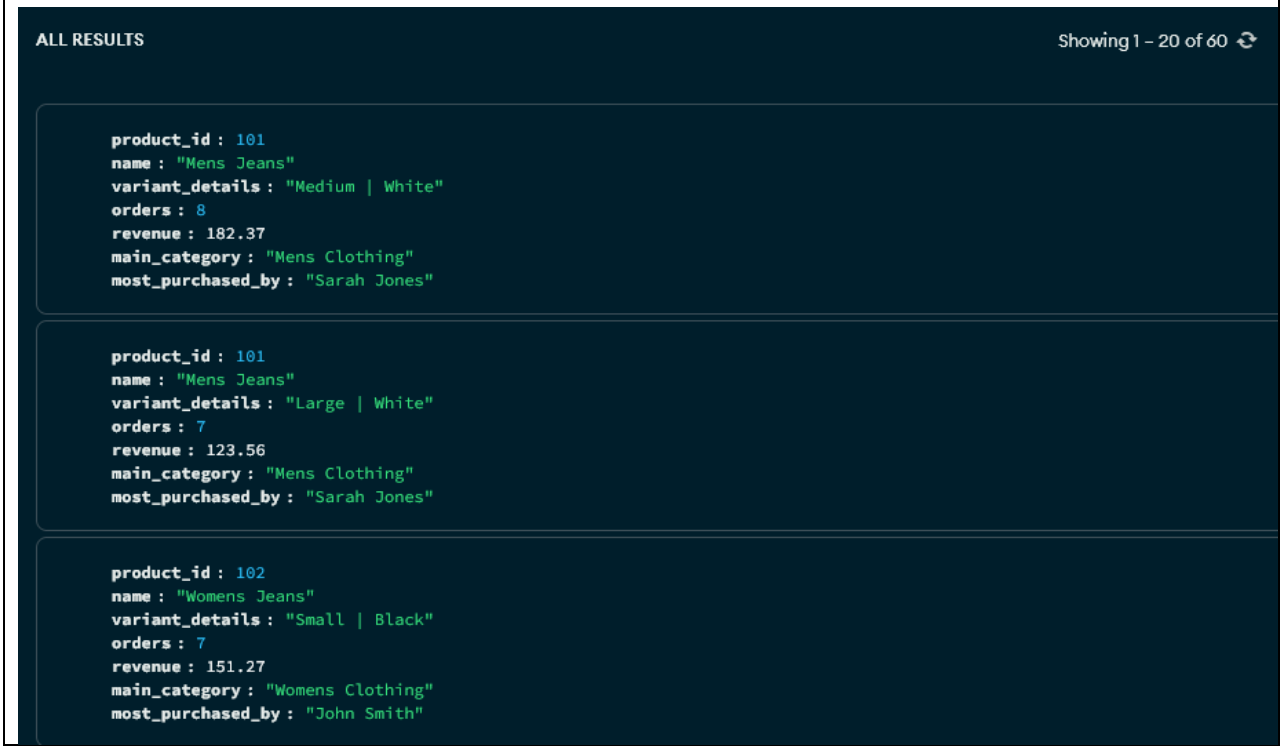
Screenshots

I've only included the screenshot of the first 6 rows of the result and the total number of rows as well.

	id integer	name character varying (100)	variant_details text	orders bigint	revenue numeric	main_category character varying (25)	most_purchased_by text
1	101	Mens Jeans	Medium | White	8	182.37	Mens Clothing	Sarah Jones
2	101	Mens Jeans	Large | White	7	123.56	Mens Clothing	Sarah Jones
3	102	Womens Jeans	Small | Black	7	151.27	Womens Clothing	John Smith
4	102	Womens Jeans	Medium | Black	7	122.13	Womens Clothing	Thomas Anderson
5	102	Womens Jeans	Large | Black	7	128.23	Womens Clothing	Sarah Jones
6	103	Mens Shirt	Medium | White	7	63.83	Mens Clothing	Emma Brown

Total rows: 60 Query complete 00:00:00.310

I've included the screenshot of the first 3 documents from the results, which also includes the total number of documents in the results



Query b: A query which uses one (or more) of the UNION, DIFFERENCE or INTERSECT operators.

This query emulates an inventory export that can be used by the procurement team to order new stock or given to a dropshipper that wishes to resell the products on their website. It has all the details of each variant.

In this query, the variants are unnested and cross-joined to list all the variants of a product table. This is done for all the products tables, and they are joined with a UNION ALL between each table.

In NoSQL, it is done in just 2 stages, as all the products are stored in one table. If a type-specific field does not exist, it is not listed.

Exporting the results of both gives the exact same CSV file with a minor difference, i.e. the missing fields are empty in the NoSQL export but have the value NULL in the SQL report.

SQL code

```
SELECT
v.sku AS sku,
p.name AS product,
p.base_price + v.price_adjustment AS price,
CONCAT(v.size, ' | ', v.color) AS variant_details,
v.stock_quantity AS stock,
p.fabric_type AS material,
NULL AS warranty,
NULL AS voltage,
NULL AS made_for
FROM clothing_products p
CROSS JOIN unnest(p.variants) v

UNION ALL

SELECT
v.sku AS sku,
p.name AS product,
p.base_price + v.price_adjustment AS price,
CONCAT(v.size, ' | ', v.color) AS variant_details,
v.stock_quantity AS stock,
NULL AS fabric,
```

```
p.warranty_period AS warranty,
p.voltage_requirements AS voltage,
NULL AS made_for
FROM electronics_products p
CROSS JOIN unnest(p.variants) v

UNION ALL

SELECT
v.sku AS sku,
p.name AS product,
p.base_price + v.price_adjustment AS price,
CONCAT(v.size, ' | ', v.color) AS variant_details,
v.stock_quantity AS stock,
p.material AS material,
NULL AS warranty,
NULL AS voltage,
p.pet_type AS made_for
FROM pet_products p
CROSS JOIN unnest(p.variants) v;
```

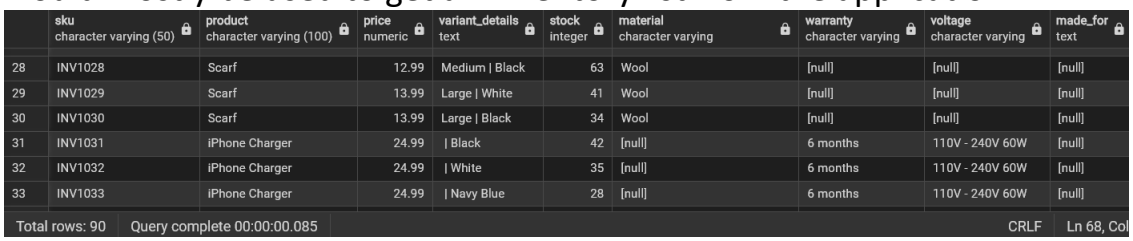
MongoDB code

```
db.products.aggregate([
  { $unwind: {
      path: "$variants",
      preserveNullAndEmptyArrays: true
    }},
  { $project: {
      _id: 0,
      sku: "$variants.sku",
      product: "$name",
      price: { $round: [ { $add: [ "$base_price", "$variants.price_adjustment" ]}, 2 ] },
      variant_details: { $cond: { if: "$variants.size", then: { $cond: { if:
"$variants.color", then: { $concat: [ "$variants.size", " | ", "$variants.color" ]},
else: { $concat: [ "$variants.size", " | " ]}}}, else: { $concat: [" | ",
"$variants.color"]}}},
      stock: "$variants.stock_quantity",
      material: { $cond: { if: "$fabric_type", then: "$fabric_type", else: "$material"
}},
      warranty: "$warranty_period",
      voltage: "$voltage_requirements",
      made_for: "$pet_type"
```

```
    }
  }
]);
```

Screenshots

I've only included a screenshot of a few rows, but also a screenshot of an exported CSV file (with folded rows to display every type of product) as this query would mostly be used to get an inventory list from the application:



	sku character varying (50)	product character varying (100)	price numeric	variant_details text	stock integer	material character varying	warranty character varying	voltage character varying	made_for text
28	INV1028	Scarf	12.99	Medium | Black	63	Wool	[null]	[null]	[null]
29	INV1029	Scarf	13.99	Large | White	41	Wool	[null]	[null]	[null]
30	INV1030	Scarf	13.99	Large | Black	34	Wool	[null]	[null]	[null]
31	INV1031	iPhone Charger	24.99	| Black	42	[null]	6 months	110V - 240V 60W	[null]
32	INV1032	iPhone Charger	24.99	| White	35	[null]	6 months	110V - 240V 60W	[null]
33	INV1033	iPhone Charger	24.99	| Navy Blue	28	[null]	6 months	110V - 240V 60W	[null]

Total rows: 90 Query complete 00:00:00.085 CRLF Ln 68, Col

CSV export:

	A	B	C	D	E	F	G	H	I
1	sku	product	price	variant_details	stock	material	warranty	voltage	made_for
2	INV1001	Mens Jeans	14.99	Small | White	47	Jeans	NULL	NULL	NULL
3	INV1002	Mens Jeans	14.99	Small | Black	38	Jeans	NULL	NULL	NULL
30	INV1029	Scarf	13.99	Large | White	41	Wool	NULL	NULL	NULL
31	INV1030	Scarf	13.99	Large | Black	34	Wool	NULL	NULL	NULL
32	INV1031	iPhone Charger	24.99	| Black	42	NULL	6 months	110V - 240V 60W	NULL
33	INV1032	iPhone Charger	24.99	| White	35	NULL	6 months	110V - 240V 60W	NULL
60	INV1059	Smartwatch	19.99	| Rose Gold	48	NULL	12 months	5V 1A	NULL
61	INV1060	Smartwatch	19.99	| Dark Grey	32	NULL	12 months	5V 1A	NULL
62	INV1061	Cat Food	4.99	Kitten 0.5 kg |	0	Fish, Chicken, Beef, Vegetables	NULL	NULL	Cats
63	INV1062	Cat Food	4.99	Adult 0.5 kg |	0	Fish, Chicken, Beef, Vegetables	NULL	NULL	Cats
90	INV1089	Chew Toy	3.99	Medium Fish |	0	Natural Rubber	NULL	NULL	Dogs, Cats
91	INV1090	Chew Toy	4.99	Large Fish |	0	Natural Rubber	NULL	NULL	Dogs, Cats

I've included a few documents from the results with the total count of the results. However, a better comparison is the CSV export, so I have also included that.



ALL RESULTS Showing 21 – 40 of 90

```
    }
  {
    "sku": "INV1029",
    "product": "Scarf",
    "price": 13.99,
    "variant_details": "Large | White",
    "stock": 41,
    "material": "Wool"
  }
  {
    "sku": "INV1030",
    "product": "Scarf",
    "price": 13.99,
    "variant_details": "Large | Black",
    "stock": 34,
    "material": "Wool"
  }
  {
    "sku": "INV1031",
    "product": "iPhone Charger",
    "price": 24.99,
    "variant_details": " | Black",
    "stock": 42,
    "warranty": "6 months",
    "voltage": "110V - 240V 60W"
  }
```

CSV export:

	A	B	C	D	E	F	G	H	I
1	sku	product	price	variant_details	stock	material	warranty	voltage	made_for
2	INV1001	Mens Jeans	14.99	Small | White	47	Jeans			
3	INV1002	Mens Jeans	14.99	Small | Black	38	Jeans			
30	INV1029	Scarf	13.99	Large | White	41	Wool			
31	INV1030	Scarf	13.99	Large | Black	34	Wool			
32	INV1031	iPhone Charger	24.99	| Black	42		6 months	110V - 240V 60W	
33	INV1032	iPhone Charger	24.99	| White	35		6 months	110V - 240V 60W	
60	INV1059	Smartwatch	19.99	| Rose Gold	48		12 months	5V 1A	
61	INV1060	Smartwatch	19.99	| Dark Grey	32		12 months	5V 1A	
62	INV1061	Cat Food	4.99	Kitten 0.5 kg |	0	Fish, Chicken, Beef, Vegetables			Cats
63	INV1062	Cat Food	4.99	Adult 0.5 kg |	0	Fish, Chicken, Beef, Vegetables			Cats
90	INV1089	Chew Toy	3.99	Medium Fish |	0	Natural Rubber			Dogs, Cats
91	INV1090	Chew Toy	4.99	Large Fish |	0	Natural Rubber			Dogs, Cats

Query c: A query which requires use of either an array or table inheritance

This query updates the price_adjustment field of a variant inside the variants array in the products table and the last_modified is set to the current timestamp. The specific price increment of a variant might need to be adjusted depending on the varying cost of a product if purchased from a different supplier or the increase in price over time from the manufacturer.

In SQL, this is done with a stored procedure to update pricing and a trigger to update last_modified, and then a select query is run to show the updated price on the variant. The Select query can be run before the procedure to show the old price of the variant.

In NoSQL, this is done with the updateOne method, and then an aggregation pipeline is run to display the updated price. Similar to the SQL query, the aggregation pipeline can be run before the update method to display the old price of the variant.

Both queries update the price for the product with id 305 and its first variant with id 1; the new price is set to 3.00. Of course, it can be changed manually, but will be done by passing variables in the queries from the backend in a production environment.

SQL code

```
CALL update_variant_price(305, 1, 3.00);

SELECT p.name, CONCAT(p.variants[1].size, ' | ', p.variants[1].color) AS
variant_details, p.variants[1].price_adjustment AS variant_price, p.last_modified
FROM products p
WHERE p.product_id = 305;
```

MongoDB code

```
db.products.updateOne({
   product_id: 305,
```

```
    "variants.variant_id": 1
  },
  { $set:{
     "variants.$.price_adjustment": 3.00,
          last_modified: new Date()
    }
  });

db.products.aggregate([
  { $unwind: {
     path: "$variants",
     preserveNullAndEmptyArrays: true
    }},
  { $match: {
     product_id: 305,
     "variants.variant_id": 1
    }},
  { $project: {
     _id: 0,
     name: "$name",
     variant_details: { $cond: { if: "$variants.size", then: { $cond: { if:
"$variants.color", then: { $concat: [ "$variants.size", " | ", "$variants.color" ]},
else: { $concat: [ "$variants.size", " | " ]}}}, else: { $concat: [" | ",
"$variants.color"]}}},
     variant_price: "$variants.price_adjustment",
     last_modified: "$last_modified"
    }
  }
]);
```

Screenshots

Old pricing and timestamp by running the SELECT query below the price update query:

	name character varying (100)	variant_details text	variant_price numeric (10,2)	last_modified timestamp without time zone
1	Chew Toy	Small Bone |	0.00	2025-05-16 16:40:15

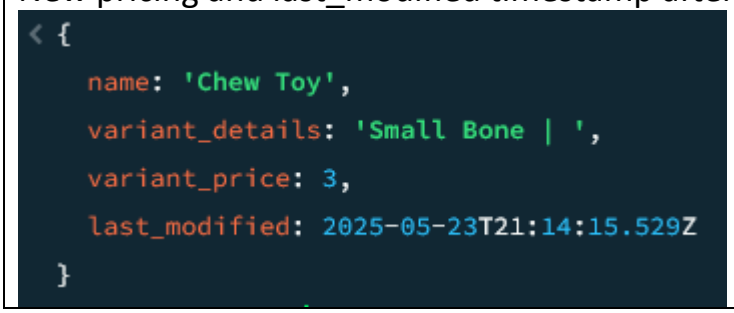
New pricing and last_modified timestamp after running the price update query and the select query to display the updated fields:

	name character varying (100)	variant_details text	variant_price numeric (10,2)	last_modified timestamp without time zone
1	Chew Toy	Small Bone |	3.00	2025-05-23 22:11:17.583792

Old pricing by just running the aggregate pipeline below the update query:



New pricing and last_modified timestamp after running the entire code block:



Query d: A query using temporal features (e.g., timestamps, intervals, etc.)

This query shows all the orders in the last 60 days with all the related details fetched from referenced entities. This would be used on an orders page that displays all orders. There would be a time interval constraint by default to not load too many orders at once and slow down the page load time. The time interval is hard-coded as 60 days (which can be manually changed in the queries), but it would be passed as a variable in the backend implementation.

SQL code

```
SELECT
o.order_id,
CONCAT(u.forenames, ' ' ,u.last_name) AS customer,
p.name AS product,

CONCAT(p.variants[(o.item).variant_id].size, ' | ',
p.variants[(o.item).variant_id].color) AS variant_details,

(o.item).quantity,
(o.item).unit_price,
(o.item).discount_amount AS discount,
o.total_amount AS order_total,
(o.shipping_address).recipient_name AS recipient,
(o.shipping_address).street AS street_address,
```

```
(o.shipping_address).city AS city,
(o.shipping_address).zip_code AS postal_code,
(o.shipping_address).country AS country
FROM orders o
JOIN users u ON o.user_id = u.user_id
JOIN products p ON (o.item).product_id = p.product_id
WHERE o.created_at > CURRENT_TIMESTAMP - interval '60 days'
ORDER BY o.order_id;
```

MongoDB code

```
db.orders.aggregate([
  { $match: {
    $expr: { $gt: [ "$created_at", { $dateSubtract: { startDate: "$$NOW", unit:
"day", amount: 60 }}]}
  }},
  { $lookup: {
     from: "products",
     localField: "item.product_id",
     foreignField: "product_id",
     let: { vid: { $subtract: ["$item.variant_id", 1]}},
     pipeline: [
      { $project: {
         _id: 0,
         name: "$name",
         variant_size: {
         $arrayElemAt: [ "$variants.size", "$$vid"]},
         variant_color: { $arrayElemAt: [ "$variants.color", "$$vid" ]}
        }}],
     as: "product"
   }},
  { $unwind: {
     path: "$product",
     preserveNullAndEmptyArrays: true
   }},
  { $project: {
       _id: 0,
      order_id: "$order_id",
      customer: "$user",
      product: "$product.name",
      variant_details: { $cond: { if: "$variants.size", then: { $cond: { if:
"$variants.color", then: { $concat: [ "$variants.size", " | ", "$variants.color" ]},
```

```
else: { $concat: [ "$variants.size", " | " ]}}}, else: { $concat: [" | ",
"$variants.color"]}}},
      quantity: "$item.quantity",
      unit_price: "$item.unit_price",
      discount: "$item.discount_amount",
      order_total: "$total_amount",
      recipient: "$shipping_address.recipient_name",
      street_address: "$shipping_address.street",
      city: "$shipping_address.city",
      postal_code: "$shipping_address.zip_code",
      country: "$shipping_address.country"
    }
  }
]);
```

Screenshots

At the time of running this query, there were 53 orders in the last 60 days. The screenshot has only a few records displayed from the result but the total count can be seen on the bottom.

	order_id integer	customer text	product character varying (100)	variant_details text	quantity integer	unit_price numeric (10,2)	discount numeric (10,2)	order_total numeric (12,2)	recipient character varying (25)	street_address character varying (100)	city character varying (60)	postal_code character varying (10)	country character varying (56)
1	148	William Moore	Womens Shirt	Large | Black	2	15.99	3.75	28.23	Charlotte Moore	36 Marylebone High Street	London	W1U 4QE	UK
2	149	Charlotte Thomas	Scarf	Small | White	1	11.99	0.00	11.99	Charlotte Thomas	17 Pall Mall	London	SW1Y 5LA	UK
3	150	Charlotte Thomas	iPhone Charger	| White	1	24.99	4.90	20.09	Charlotte Thomas	17 Pall Mall	London	SW1Y 5LA	UK
4	151	Daniel White	Scarf	Large | White	1	11.99	0.00	11.99	Daniel White	4 St James's Square	London	SW1Y 4JU	UK
5	152	James Johnson	iPhone Charger	| Dark Grey	1	24.99	4.60	20.39	James Johnson	17 Fleet Street	London	EC4Y 1AA	UK
6	153	Daniel White	Smartwatch	| White	1	19.99	0.00	19.99	Daniel White	4 St James's Square	London	SW1Y 4JU	UK

Total rows: 53 | Query complete 00:00:00.081 | CRLF | Ln 105,

The MongoDB query also returns the same results

ALL RESULTS — Showing 1 – 20 of 53


```
order_id : 151
customer : "Daniel White"
product : "Scarf"
variant_details : "Large | White"
quantity : 1
unit_price : 11.99
discount : 0.00
order_total : 11.99
recipient : "Daniel White"
street_address : "4 St James's Square"
city : "London"
postal_code : "SW1Y 4JU"
country : "UK"
```

```
order_id : 152
customer : "James Johnson"
product : "iPhone Charger"
variant_details : " | Dark Grey"
quantity : 1
unit_price : 24.99
discount : 4.60
order_total : 20.39
recipient : "James Johnson"
street_address : "17 Fleet Street"
city : "London"
postal_code : "EC4Y 1AA"
country : "UK"
```

```
order_id : 153
customer : "Daniel White"
product : "Smartwatch"
variant_details : " | White"
quantity : 1
unit_price : 19.99
discount : 0.00
order_total : 19.99
recipient : "Daniel White"
street_address : "4 St James's Square"
```

Query e: A query using OLAP (e.g., ROLLUP, CUBE, PARTITION) features

This query shows the total number of orders and the accumulated revenue from a user, broken down into years and months. This would usually be shown on the user profile in the admin dashboard to see how much a user has spent. It is crucial to know who the best buyer is in order to retain their business.

SQL code

```
SELECT
u.user_id AS id,
CONCAT(u.forenames, ' ' ,u.last_name) AS name,
EXTRACT(YEAR FROM o.created_at) AS year,
EXTRACT(MONTH FROM o.created_at) AS month,
count(o.*) AS orders,
sum(o.total_amount) AS revenue
FROM
users u
JOIN orders o ON u.user_id = o.user_id
WHERE u.user_id = 2
GROUP BY id, ROLLUP(year, month)
ORDER BY year DESC, month DESC;
```

MongoDB code

```
db.users.aggregate([
 { $match: {
      user_id: 2
    }},
 { $lookup: {
    from: "orders",
    localField: "user_id",
    foreignField: "user_id",
    pipeline: [ {
       $project: {
         _id: 0,
         revenue: "$total_amount",
         placed: "$created_at"
       }}
    ],
  as: "rorders"}},
 { $unwind: {
```

```
    path: "$rorders",
    preserveNullAndEmptyArrays: true
  }},
{ $group: {
     _id: {
      id: "$user_id",
      name: { $concat: [ "$forenames", " ", "$last_name" ]},
      year: { $year: "$rorders.placed" },
      month: { $month: "$rorders.placed" }
     },
     orders: { $sum: 1 },
     revenue: { $sum: "$rorders.revenue" }
  }},
{ $sort: {
     "_id.year": -1,
     "_id.month": -1
  }},
{ $facet: {
   total_stats: [
    { $group: {
        _id: {
         id: "$_id.id",
         name: "$_id.name"
        },
        total_orders: { $sum: "$orders" },
        total_revenue: { $sum: "$revenue" }
       }},
    { $project: {
        _id: "$_id.id",
        name: "$_id.name",
        total_orders: "$total_orders",
        total_revenue: "$total_revenue"
       }}
   ],
   detailed_stats: [
    { $group: {
        _id: { year: "$_id.year" },
        yearly_orders: { $sum: "$orders" },
        yearly_revenue: { $sum: "$revenue" },
        monthly_data: {
         $push: {
```

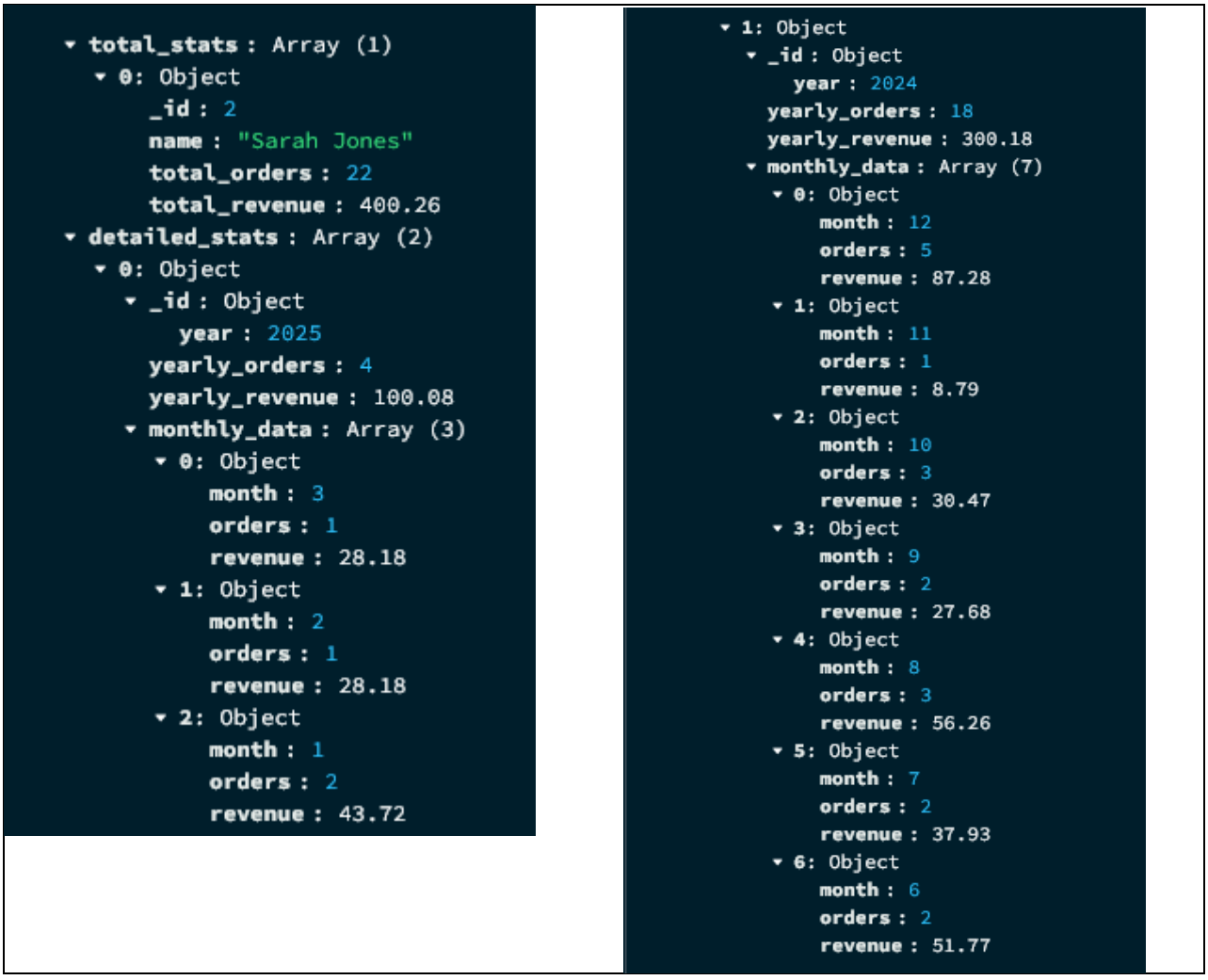
```
            month: "$_id.month",
            orders: "$orders",
            revenue: "$revenue"
          }
        }
      }},
    { $sort: {
      "_id.year": -1
      }}
  ]
 }}
]);
```

Screenshots

Here is the screenshot of the total revenue by the user in the PostgreSQL. It is broken down by year and then by month.

	id integer	name text	year numeric	month numeric	orders bigint	revenue numeric
1	2	Sarah Jones	[null]	[null]	22	400.26
2	2	Sarah Jones	2025	[null]	4	100.08
3	2	Sarah Jones	2025	3	1	28.18
4	2	Sarah Jones	2025	2	1	28.18
5	2	Sarah Jones	2025	1	2	43.72
6	2	Sarah Jones	2024	[null]	18	300.18
7	2	Sarah Jones	2024	12	5	87.28
8	2	Sarah Jones	2024	11	1	8.79
9	2	Sarah Jones	2024	10	3	30.47
10	2	Sarah Jones	2024	9	2	27.68
11	2	Sarah Jones	2024	8	3	56.26
12	2	Sarah Jones	2024	7	2	37.93
13	2	Sarah Jones	2024	6	2	51.77

Here are the screenshots of the MongoDB result. It is also broken down into years and then months within the years. There are multiple screenshots as the result was a bit too long in Compass. It is not exactly in the same format, but it is providing the exact same data as the SQL query.



References:

Strausz, P. (2023) *[Part3] PostgreSQL Relationships: Creating and Managing Database Relationships.* Available at: https://medium.com/@patrikstrausz17/part3-postgresql-relationships-creating-and-managing-database-relationships-5e1eaea769d8#:~:text=To%20implement%20a%20many%2Dto%2Dmany%20relationship%20in%20PostgreSQL (Accessed: 13 May, 2025)

MongoDB Docs (no date) *Reduce $lookup Operations.* Available at: https://www.mongodb.com/docs/atlas/schema-suggestions/reduce-lookup-operations/#:~:text=slow%20and%20resource%2Dintensive (Accessed: 19 May 2025)

Gupta, A. (2024) *Advanced Techniques with MongoDB: Mastering Lookup and Unwind.* Available at: https://medium.com/@akshatgupta1903/advanced-techniques-with-mongodb-mastering-lookup-and-unwind-acfc8a8ad5b9#:~:text=Indexing%20is%20a%20critical%20factor (Accessed: 19 May 2025)